HOTMOBILE 2027

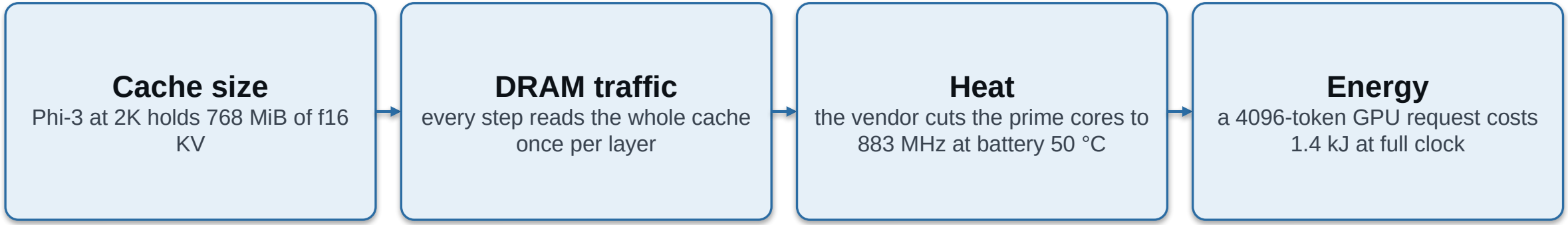
Evicting Makes It Slower: Three Properties a KV Cache Policy Needs on a Phone

Five published eviction policies, run in their own configurations on a OnePlus 15.
None behaves as published. Two engine properties explain it, and a third decides quality.

Md Romyull Islam · Kennesaw State University
OnePlus 15, Snapdragon 8 Elite Gen 5, Adreno 840 · every number measured on the device

THE PROBLEM

The phone is bound by the cache, not the matrix multiply



One chain. Cache size sets traffic, traffic sets power, power sets temperature, temperature sets the throttle, and the throttle sets throughput.
So a mobile cache policy has to be judged on quality, throughput, heat and energy at once, on real hardware.

Every policy below was built for a server GPU and judged by perplexity alone.

THE THESIS

Three properties decide the outcome. Every published policy lacks one.

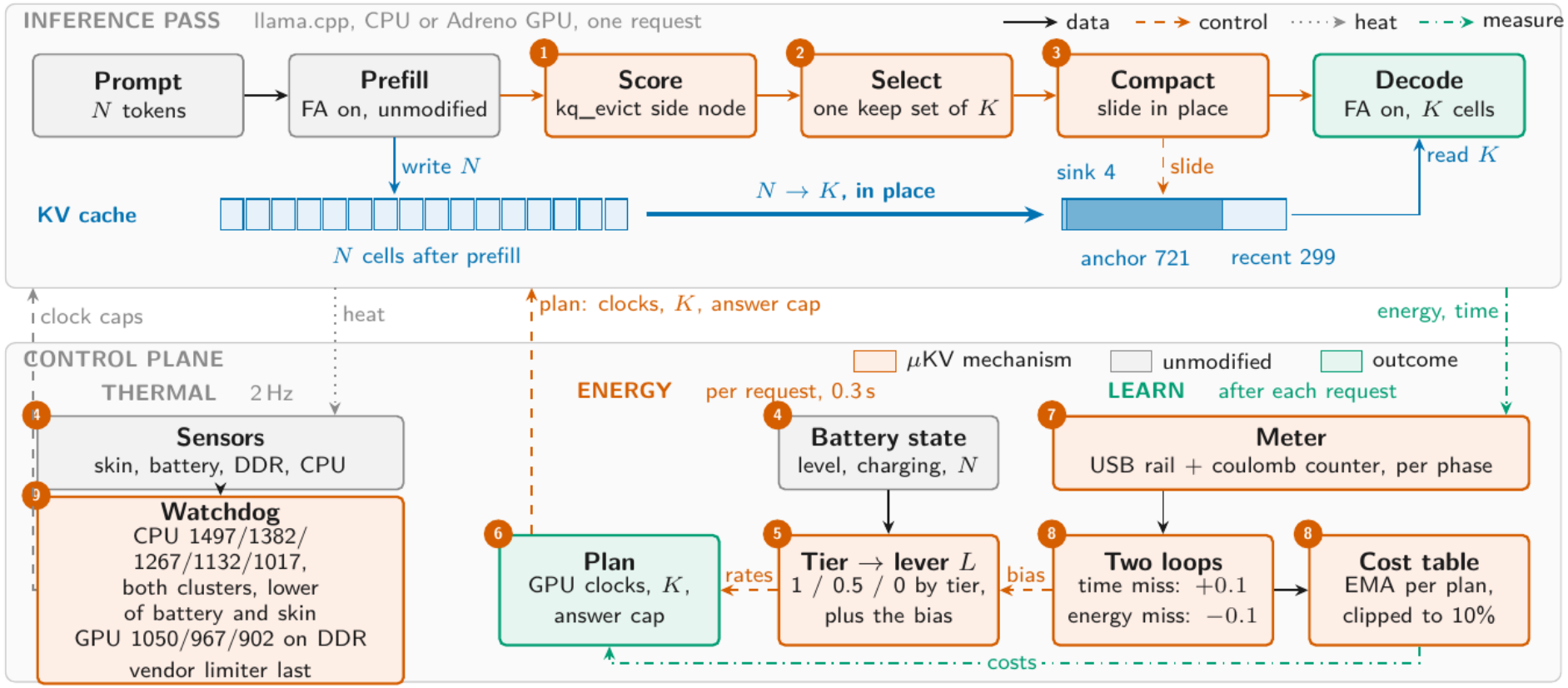
Policy class	Budget frees cells	Keeps the fused kernel	Retrieval survives
SnapKV, Ada-KV, H2O, TOVA	no, 36 to 68% held	no, 0.12 to 0.20x	yes
StreamingLLM	yes, 20.6% held	yes, 1.19x	no, 2 to 3 of 14 needles
µKV	**yes, 7.4% held**	**yes, 1.19x**	**yes, ties the full cache**

- Sequence-level selection, or the shared cell array frees nothing.
- No attention-score read at decode, or FlashAttention is lost and eviction makes the phone slower.
- Content-aware selection, or the needles go with the middle of the prompt.

µKV is the existence proof that one policy can hold all three at once. It is not a new selection rule.

DESIGN

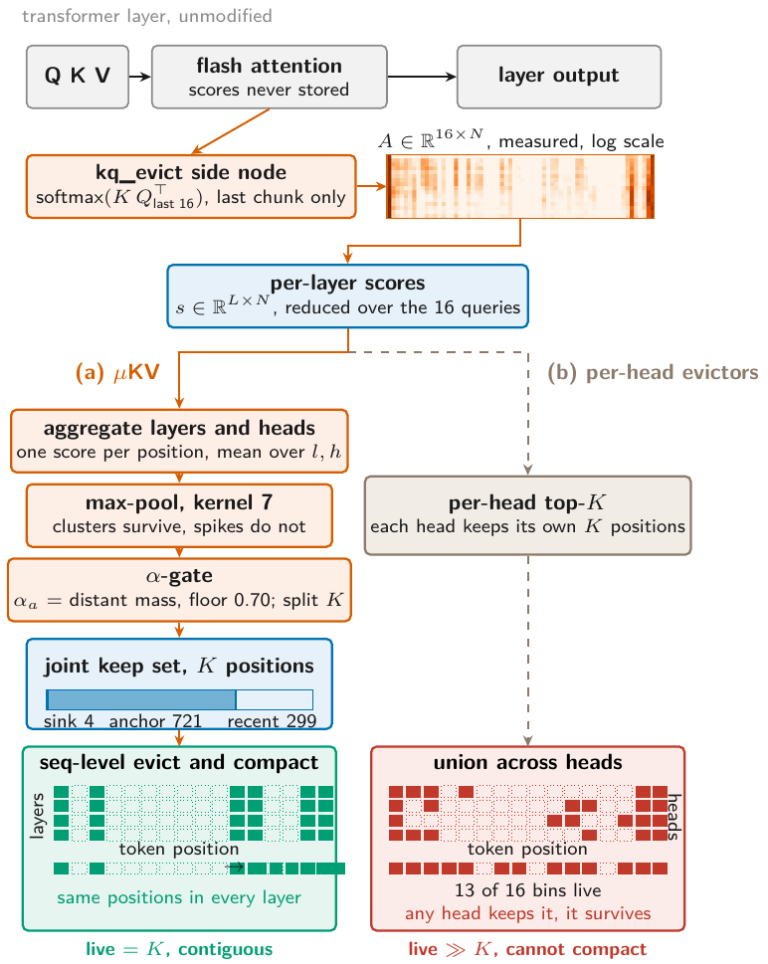
μKV on the phone: three changes to the pass, one controller beside it



Circled numbers are shared with the control-plane figure: 1-3 the pass, 4 sense, 5 decide, 6 act, 7 measure, 8 learn, 9 thermal guard. Same part, same number, both figures.

MECHANISM

Selection is sequence-level, from real attention captured in-graph



- Mean over layers and heads, then max-pool over 7 positions.
- A per-prompt split αa sets anchors against the recent window, floored at 0.70.
- On the 9737-token prompt: 4 sinks, 721 anchors, 299 recent = K of 1024.
- One keep set for every head and layer, which is what lets the array compact.
- Heat map is a real capture on Llama-3.2-1B, layer 8, 16 queries.

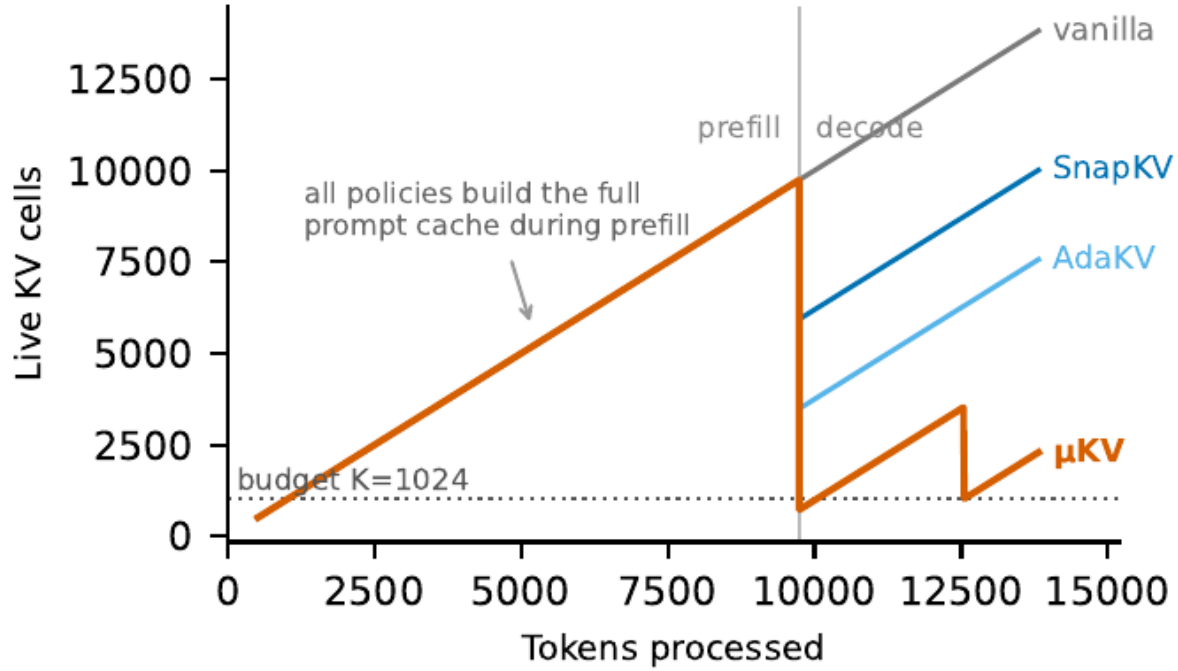
Every stage in order, and what each one produces

	Stage	What it does	What comes out	Fig 1
a	flash attention	fuses softmax into the kernel, so the attention matrix is never written to memory	layer output only	—
b	kq_evict side node	recomputes softmax(K Q^T) for the last 16 queries, on the last prefill chunk only, as a graph output rather than a mid-graph read	A, 16 x N scores	1
c	reduce over queries	averages the 16 query rows away	s, L x N: one score per layer per position	1
d	aggregate layers and heads	mean over l and h. This is the step that makes selection sequence-level	one score per position, length N	2
e	max-pool, kernel 7	each position takes the max of its 7-wide neighbourhood, so clusters survive and isolated spikes do not	smoothed score, length N	2
f	α-gate	measures αa, the attention mass outside the recent window, floored at 0.70 and capped at 0.95, then splits the K budget	n_anchor and n_recent	2
g	select	takes the top n_anchor positions by score, plus 4 sink tokens, plus the recent window	joint keep set, K positions	2
h	compact in place	survivors slide into a dense prefix inside the tensors prefill already allocated	live = K, contiguous	3

Measured instance: on the 9737-token prompt αa came out at 0.71, so K = 1024 is 4 sinks, 721 anchors and 299 recent cells.

A per-head evictor skips stage d. Each head then keeps its own K, the union across heads leaves live $\gg$ K, and the array cannot compact.

Every policy builds the full prompt cache. Only some give it back.



- One cell array serves every head and every layer.
- A cell is freed only when every head and every layer drops it.
- SnapKV bottoms out at 5.9K live cells against a nominal 1024.
- µKV compacts to 721 and its re-eviction holds near 2K.

TABLE 1 · SUSTAINED DECODE ON THE PHONE CPU

9737-token prompt, 4096 generated, matched budget 1024

Policy	tok/s	Wall (s)	Live cells	DDR °C	mWh
vanilla (full cache)	5.0	1055	9737	59.8	1033
µKV	**24.2**	**406**	**721 (1980)**	**54.4**	**391**
SnapKV	6.8	902	5929	56.3	818
Ada-KV	6.7	875	3484	57.1	1002
StreamingLLM	6.6	877	777	56.7	1052
H2O	5.8	1006	6110	54.8	1094
TOVA	6.0	947	4091	54.4	1069

Llama-3.2-1B

µKV buys 4.8x the throughput at 7% of the cells and 62% less energy, with a DDR peak 5 °C below vanilla.
The four per-head evictors run at 5.8 to 6.8 tok/s and 818 to 1094 mWh against vanilla's 1033. That spread straddles the full cache rather than beating it.

Caveat: this campaign ran StreamingLLM without the fused-kernel flag, so its row understates it. See the GPU correction.

TABLE 1 continued · THREE MORE MODELS, EACH BASELINE AT ITS OWN BUDGET

Bonsai-8B (1-bit weights), Phi-3-mini and gemma-2-2b. µKV at K = 1024 throughout.

Model	vanilla tok/s	µKV tok/s	gain	vanilla mWh	µKV mWh	energy cut	µKV cells
Bonsai-8B	1.0	4.5	4.5x	5758	2312	60%	789 (1993)
Phi-3-mini	1.2	5.5	4.5x	2046	926	55%	929
gemma-2-2b	3.9	9.3	2.4x	974	589	40%	804

At their own published budgets the gap widens: SnapKV at 2048 per head keeps 94% of the cache, H2O at 20% of the prompt keeps 92%, TOVA and Ada-KV keep 77% and 71%. µKV keeps 14%.
On Bonsai-8B the full cache reaches a 72.2 °C DDR peak. µKV reaches 62.5 °C, 9.7 °C cooler, with no change to the model's parameters.

What the papers claim, and what the phone gives back

Policy	Its own paper claims	At its own budget on the phone	Cache actually kept
SnapKV	8.2x smaller cache at 16K	2048 per head	94%
H2O	5 to 10x at a 20% budget	20% of the prompt	92%
TOVA	per-layer budget	2048 per layer	77%
Ada-KV	adaptive per-head budget	2048	71%
μKV	**this work**	**1024, sequence-level**	**14%**

The claim is not wrong on a server. It is a claim about how many scores were dropped, not about how much memory came back.
On a shared cell array those are different numbers, and only the second one changes the phone's DRAM traffic.

Report realized cells, not selection ratios.

TABLE 2 · MOBILE GPU, GROUPED BY HOW EACH POLICY SELECTS

Adreno 840, Llama-3.2-1B, ctx 16384, cooled per cell

Policy	n	tok/s	dec x	Cells	% cache
vanilla (no eviction)	3	24.30	1.00	9741	100
SEQUENCE-LEVEL · budget frees cells, kernel survives					
µKV, K = 1024	3	29.02	1.19	723	7.4
StreamingLLM, own budget	3	29.03	1.19	2005	20.6
PER-HEAD · budget frees nothing, kernel is lost					
SnapKV	1	4.76	0.20	6592	67.7
H2O	1	3.58	0.15	6110	62.7
TOVA	1	4.34	0.18	4125	42.4
Ada-KV	1	2.86	0.12	3519	36.1
CONTROLS · one property removed at a time					
µKV, compaction off	3	25.61	1.05	723	7.4
StreamingLLM, forced off the kernel	3	5.81	0.24	777	8.0

On this phone, four of five published policies decode slower than not evicting at all.

AN ERROR WE FOUND IN OUR OWN HARNESS

StreamingLLM was run without the fused-kernel flag

Arm	tok/s	vs vanilla	Cache held	Needles
vanilla	24.30	1.00	304.4 MB	13 / 14
μKV, K = 1024	29.02	1.19	22.6 MB	13 / 14
StreamingLLM, own budget 2004	29.03	1.19	62.6 MB	3 / 14
StreamingLLM, as we first ran it	5.81	0.24	24.3 MB	invalid output

- StreamingLLM's reference implementation concatenates survivors, so compaction is intrinsic to it. Our harness removed it.
- Corrected, it ties μKV on throughput. "μKV is the only policy faster than not evicting" was false and is gone.
- This makes the result stronger. A policy we did not author confirms that the axis of selection, not our design, is what matters.
- What separates μKV is budget and retrieval: 723 cells against 2005, and 13 needles against 3.

TABLE 3 · RETRIEVAL

A fact buried at one of seven depths in 4K or 8K of filler, 392 cells

Policy	Phi-3	Llama-1B	Gemma-2B	Bonsai-8B	Cells attended
vanilla (full cache)	14	13	11	14	4858
μKV	**14**	**13**	**11**	**14**	**774**
SnapKV	14	12	11	14	4237
Ada-KV	14	12	11	14	3361
TOVA	14	12	11	14	3670
H2O	14	8	10	13	4212
StreamingLLM	3	3	2	3	890

μKV matches the full cache on every model while attending to 774 cells, against 3361 to 4237 for the score-reading evictors.
StreamingLLM fails everywhere: its window evicts exactly the middle of the context. Where every policy misses a depth, the full cache misses it too.

This workload generates 64 tokens, so it is prefill-dominated and says nothing about speed or energy.

TABLE 4 · LONGBENCH, FIVE TASKS, 50 SAMPLES EACH

Llama-3.2-1B and gemma-2-2b-it

Llama-3.2-1B	HotpotQA	2WikiMQA	MultiFieldQA	Qasper	TriviaQA	Avg	Cells kept
vanilla	42.14	23.86	45.04	17.55	81.19	41.96	8687 (100%)
µKV	**40.72**	**25.53**	**43.23**	**17.06**	**83.88**	**42.08 (+0.1)**	**826 (13.0%)**
SnapKV	41.14	23.51	45.02	22.81	81.19	42.73 (+0.8)	6717 (82.4%)
Ada-KV	41.84	22.17	46.16	22.10	81.19	42.70 (+0.7)	3327 (47.3%)
TOVA	41.61	22.98	45.62	21.93	81.86	42.80 (+0.8)	3824 (53.5%)
H2O	42.68	24.04	41.87	19.35	81.20	41.83 (-0.1)	6699 (83.1%)
StreamingLLM	35.61	18.14	36.52	16.51	81.00	37.56 (-4.4)	1994 (23.0%)

gemma-2-2b-it	HotpotQA	2WikiMQA	MultiFieldQA	Qasper	TriviaQA	Avg	Cells kept
vanilla	46.51	34.51	45.19	36.42	87.67	50.06	6457 (100%)
µKV	**40.32**	**37.20**	**39.63**	**31.71**	**87.67**	**47.31 (-2.8)**	**716 (13.0%)**
SnapKV	40.51	36.81	45.40	38.38	87.44	49.71 (-0.4)	6031 (94.2%)
Ada-KV	40.51	36.81	44.55	37.26	87.44	49.31 (-0.7)	4064 (66.3%)
TOVA	41.52	37.31	44.37	37.45	87.44	49.62 (-0.4)	4087 (66.9%)
H2O	37.38	37.31	43.61	35.37	87.44	48.22 (-1.8)	5869 (92.1%)
StreamingLLM	41.51	32.67	38.72	28.59	87.67	45.83 (-4.2)	1995 (30.9%)

TABLE 4 continued

Phi-3-mini and Bonsai-8B

Phi-3-mini	HotpotQA	2WikiMQA	MultiFieldQA	Qasper	TriviaQA	Avg	Cells kept
vanilla	46.86	43.51	57.05	36.92	87.87	54.44	9702 (100%)
µKV	**45.97**	**43.18**	**55.85**	**40.80**	**86.37**	**54.43 (-0.0)**	**853 (12.2%)**
SnapKV	46.85	40.01	55.62	38.35	87.20	53.61 (-0.8)	7629 (83.4%)
Ada-KV	46.90	38.48	56.51	37.33	87.87	53.42 (-1.0)	4056 (50.6%)
TOVA	54.81	38.67	55.33	50.60	87.87	57.45 (+3.0)	4550 (54.9%)
H2O	48.32	39.66	56.12	36.97	88.20	53.85 (-0.6)	8702 (91.2%)
StreamingLLM	40.71	41.88	40.05	30.68	87.87	48.24 (-6.2)	1999 (20.6%)

Bonsai-8B	HotpotQA	2WikiMQA	MultiFieldQA	Qasper	TriviaQA	Avg	Cells kept
vanilla	35.11	33.39	50.03	38.53	86.15	48.64	8928 (100%)
µKV	**35.16**	**24.33**	**48.13**	**32.30**	**84.95**	**44.97 (-3.7)**	**797 (12.6%)**
SnapKV	38.58	34.30	50.12	38.59	88.15	49.95 (+1.3)	8716 (96.1%)
Ada-KV	34.46	34.48	50.75	37.56	87.48	48.95 (+0.3)	5293 (68.2%)
TOVA	37.06	30.19	50.76	37.88	87.48	48.67 (+0.0)	5700 (72.5%)
H2O	36.66	28.54	49.03	34.97	88.15	47.47 (-1.2)	7798 (90.9%)
StreamingLLM	34.40	24.97	30.13	30.24	86.26	41.20 (-7.4)	2246 (25.2%)

Desktop GPU, f16 KV, K = 1024; StreamingLLM at its own budget of 2004. The phone pass is on the next slide.

TABLE 5 · LONGBENCH ON THE PHONE

103 prompts on the phone CPU, Llama-3.2-1B, each policy at its own budget

Policy	Budget	2Wiki	Hotpot	Qasper	Trivia	Mean	Cache
vanilla (full cache)	full	22.79	29.29	19.96	75.35	36.85	100%
µKV	**1024**	**27.86**	**25.96**	**19.01**	**75.21**	**37.01**	**14%**
SnapKV	2048	20.50	31.43	20.72	79.19	37.96	94%
Ada-KV	2048	20.75	31.43	19.66	79.19	37.76	71%
TOVA	2048	18.75	31.37	20.33	79.19	37.41	77%
H2O	20% of N	14.61	31.32	19.58	79.19	36.17	92%
StreamingLLM	2004	22.59	22.63	18.59	76.14	34.99	33%

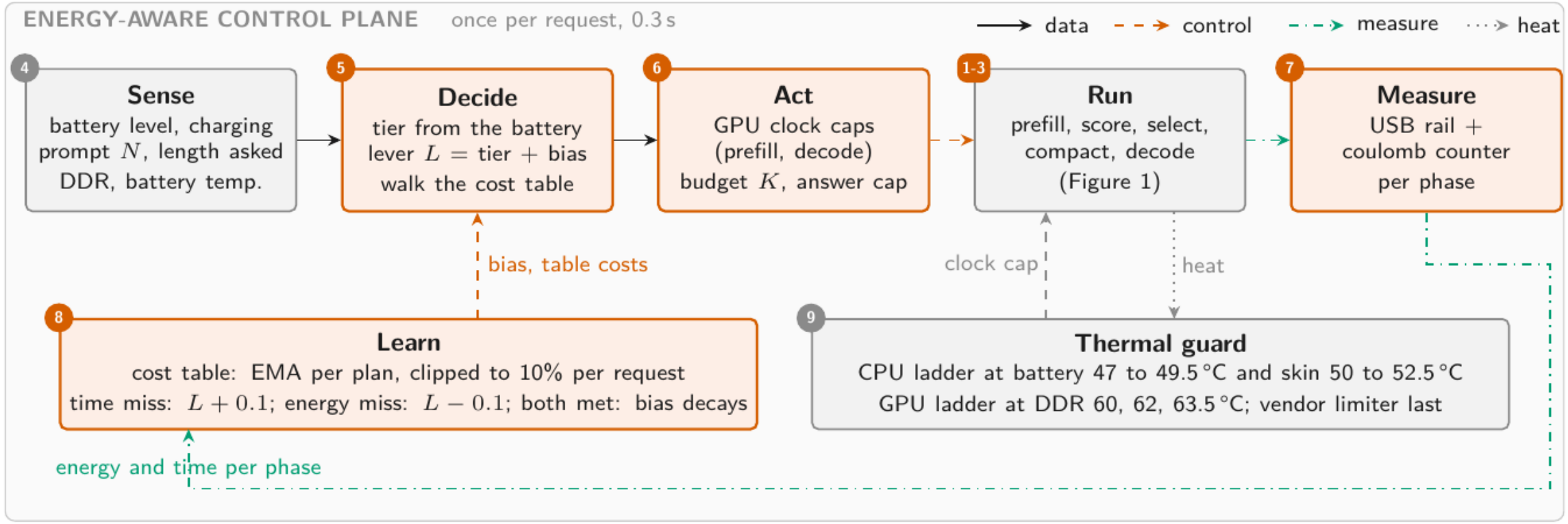
µKV scores 37.0 against the full cache's 36.9 while holding 14% of the cache.
SnapKV, Ada-KV and TOVA gain 0.6 to 1.1 F1 while holding 71 to 94%. What separates the policies is the cache each one frees, not the budget it asks for.

No. The coupling lives in the KV memory layout.

Engine	How KV memory is laid out	Can a per-head budget free memory?
llama.cpp	one contiguous cell array per layer	only where every head agrees
vLLM PagedAttention	block = [blocks, kv heads, head size, block size]	only where every head agrees
MLC-LLM	paged, the same shape	only where every head agrees
ExecuTorch, vendor SDKs	static position-indexed tensors	only where every head agrees

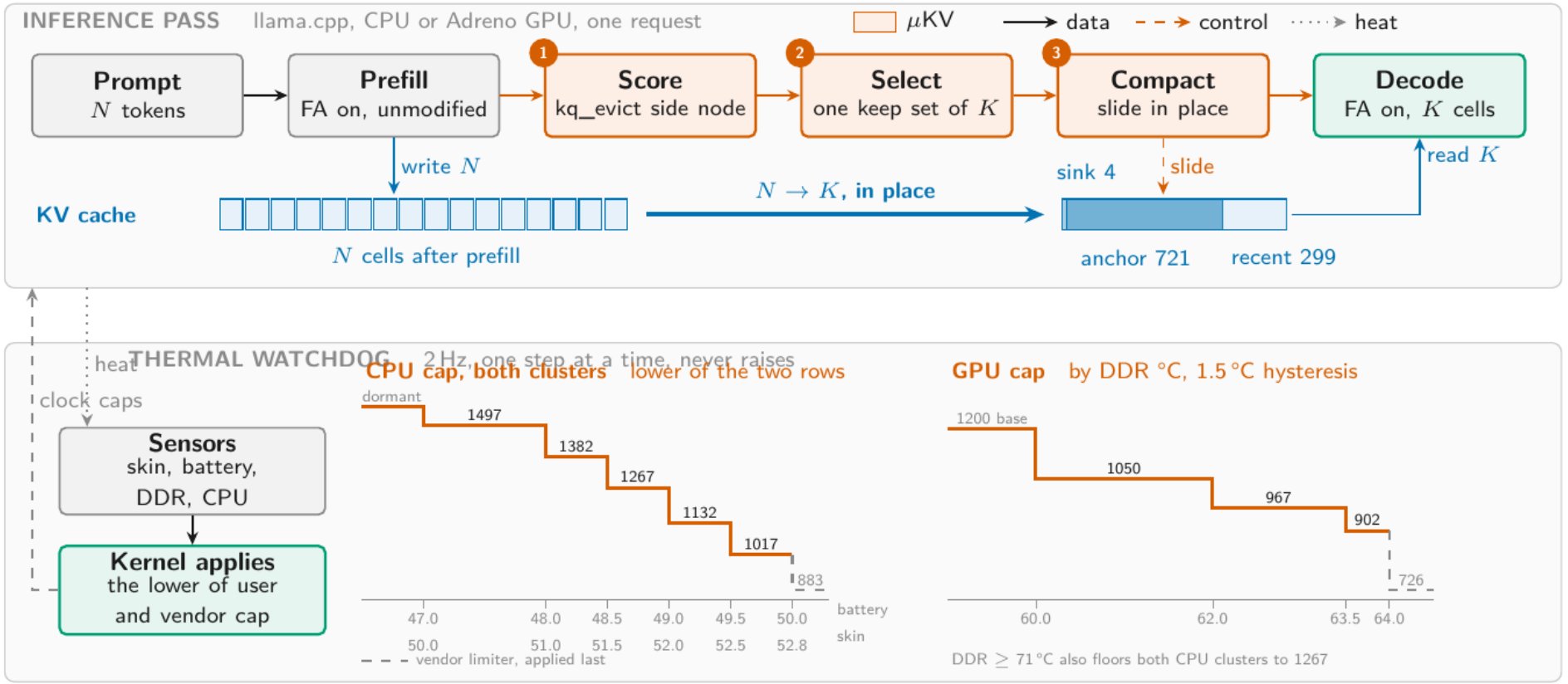
KV-Compress reports the same limitation independently: per-head eviction "adds fragmentation and cannot realize the theoretical compression rates in physical memory."
It fixes this on the server by paging per head inside PagedAttention. That needs a per-head block table and a custom kernel.
A phone runtime has neither, and a mobile GPU driver makes both expensive. That is the point.

One lever, two loops, and a cost table measured on the device



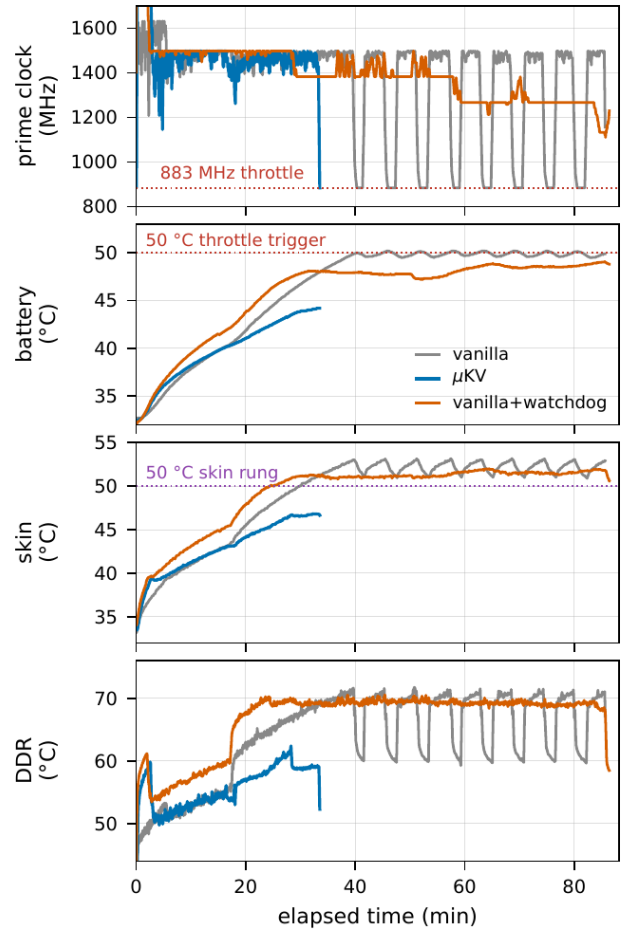
Numbers match the architecture figure, so the same part carries the same number in both. Run is steps 1 to 3. No online policy learning runs on the phone; the decision is a deterministic table walk.

Three caps, five ladders, three sensors, reduce only



Each CPU cluster takes the lower of the battery and skin ladders. DDR above 71 °C floors both to 1267 MHz. Battery here is temperature in °C, not charge in %; the scheduler tiers use charge.

Bonsai-8B, phone CPU, full cache against µKV



- The full cache runs 85.6 minutes and drives the battery to 50.2 °C.
- At 50 °C the vendor deep-throttles both clusters to 883 MHz, in a sawtooth of 17 dips totalling 923 s.
- µKV finishes in 33.2 minutes at a 44.2 °C peak, before the trigger ever fires.
- Given the watchdog, the full cache's battery rise flattens from 0.48 to 0.05 °C per minute.
- The watchdog never fired in any Table 1 cell. That result is pure eviction.

A NEGATIVE RESULT WORTH REPORTING

Cache size is not a temperature actuator

- We tied the cache budget itself to DDR temperature, a four-tier ladder over K in {256 ... 1024}.
- Halving K under co-load left peak DDR unchanged.
- On a cool phone a smaller cache runs hotter at peak, because the saved bandwidth converts straight into speed and power.
- On the GPU the clock ladder does work: 1289 against 1390 J and a DDR peak of 75 against 90 °C, for 1% more time.

Cache size controls throughput, time and energy. The clock caps heat.

TABLE 6 · ENERGY-AWARE SCHEDULING

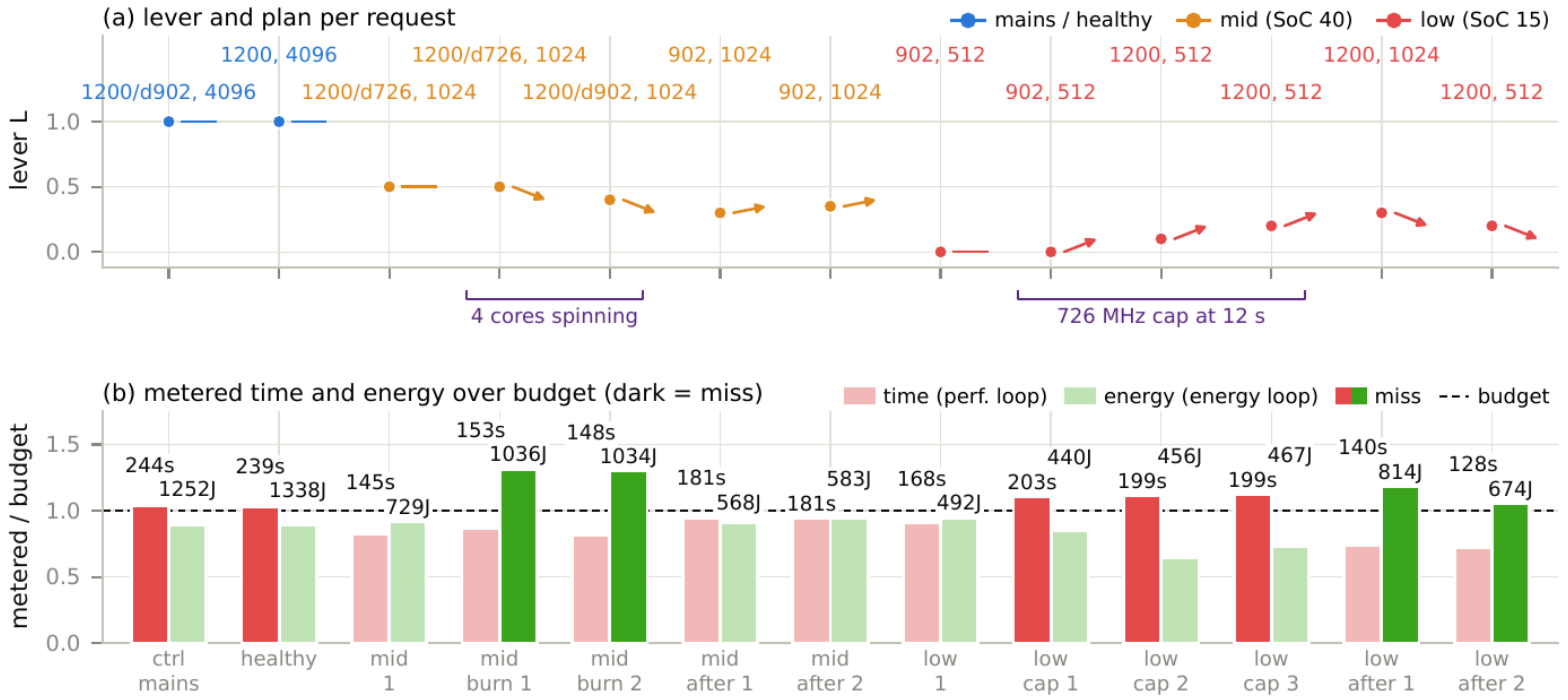
123 requests through a real discharge, 91% to 11%, charging off

Battery charge	GPU plan (MHz)	Answer cap (tokens)	Cable J	Battery J	Battery s
mains or above 50%	1200, decode 902	4096	1336	885	280
21 to 50%	1200, decode 726	1024	577	748	210
20% or below	902	512	500	506	177
GPU unavailable	CPU, K = 1024	by tier	822	—	—

The cache budget K stays at 1024 cells in every tier. The answer cap is a token count. Same number, different quantity.

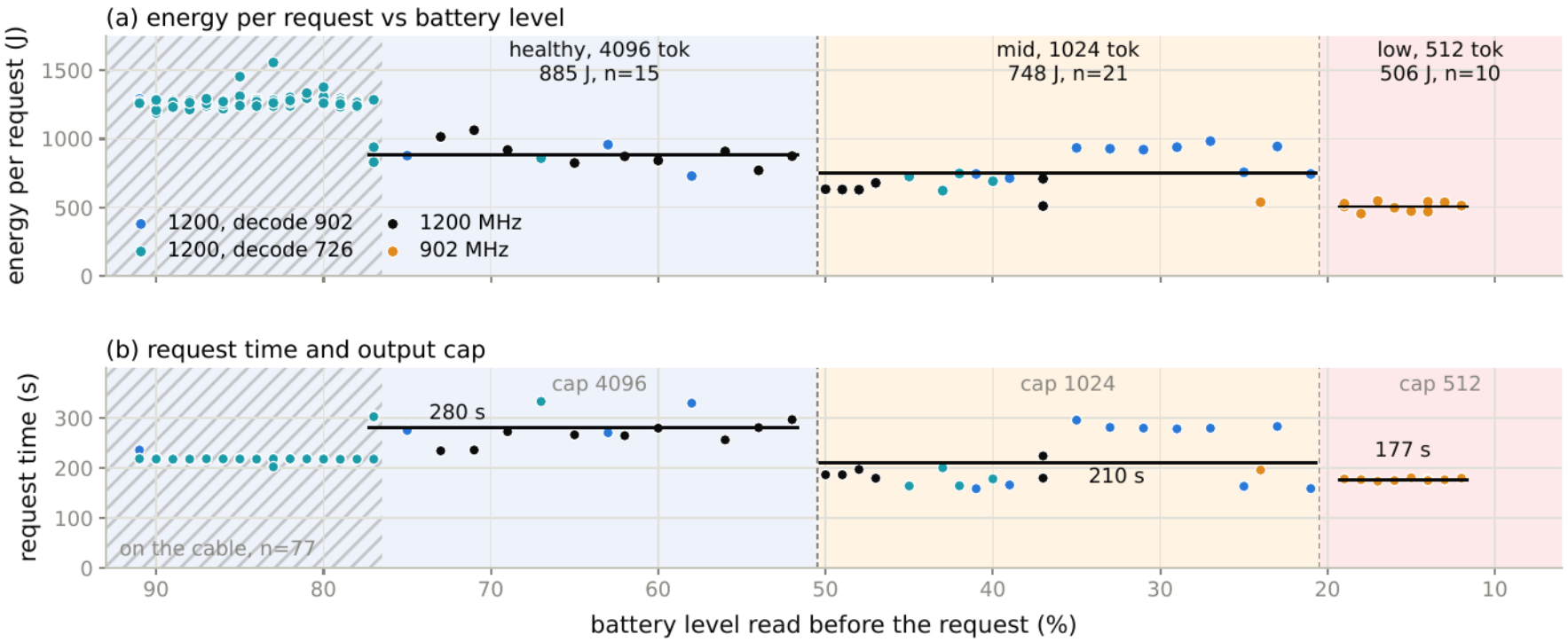
- The two lower tiers cut energy per request by 15% and 43%, with time falling 25% and 37%.
- On the battery the phone is a different machine. The same healthy request costs 885 J instead of 1336.
- The OEM caps prefill at 726 MHz within a minute of every request when unplugged, so the answer cap carries the saving.
- Mean prediction error was 8.9% on battery against 3.7% on the cable. The clipped learning tracked the shift.

Thirteen cooled requests carrying real disturbances



Four spinning cores pushed a mid-tier request 30% over budget. An external 726 MHz cap put three low-tier requests 10 to 12% over time.

A loop resident on the phone, 123 requests, no forced state



The scheduler switched on the first request past each threshold: 4096 tokens down to 52%, the 1024 cap from 50%, the low plan from 19%.

A contextual bandit on the same objective, one pull = one real request

Context	Battery	wq	wt	we	Time slack
healthy	mains or >50%	0.55	0.40	0.05	5%
mid	21 to 50%	0.35	0.20	0.45	22%
low	20% or below	0.20	0.10	0.70	40%

Reward $r = wq - wt \cdot T/Tref - we \cdot E/Eref$
with Eref = 782.0 J and Tref = 142.4 s, the cost table's prediction for this request at 1200 MHz.

- Arms: GPU clock 1200, 902, 726 MHz. Flat caps only.
- Reward is the scheduler's own tier utility, so both optimize the same thing.
- Epsilon-greedy 0.25, warm start of one pull per context and arm.
- 24 pulls, each behind a cooling gate, energy metered.
- Llama-3.2-1B, 9737 prompt, 1024 output, K = 1024.

TABLE 7 · EVERY CONTEXT AND ARM THE BANDIT PULLED

All 24 pulls, measured energy and time

Context	Arm (MHz)	pulls	Energy (J)	Time (s)	Mean reward	
healthy	**1200**	**6**	**804.6**	**139.1**	**+0.1079**	**bandit's choice**
healthy	902	1	599.4	180.6	+0.0043	
healthy	726	1	559.0	218.4	-0.0991	
mid	1200	1	799.5	139.1	-0.3054	
mid	**902**	**6**	**579.6**	**180.5**	**-0.2370**	**bandit's choice**
mid	726	1	579.5	218.5	-0.2903	
low	1200	1	790.2	139.2	-0.6050	
low	**902**	**4**	**591.4**	**180.7**	**-0.4562**	**bandit's choice**
low	726	3	563.4	218.4	-0.4577	within 0.0015 of 902

The low tier's two best arms are effectively tied, which is why the scheduler's stopping rule refusing 726 costs nothing.

TABLE 8 · WHAT LEARNING BOUGHT

Rule and bandit scored on the same objective

Tier	Rule's plan	Utility	Bandit	Utility	Gain	Verdict
healthy	prefill 1200, decode 902	+0.0989	1200	+0.0999	+0.0010	tie
mid	**prefill 1200, decode 726**	**-0.2845**	**902**	**-0.2440**	**+0.0405**	**bandit wins**
low	902	-0.4551	902	-0.4551	0	identical

The one disagreement, in physical units

Mid tier, 9737 prompt + 1024 output	Energy (J)	Time (s)	Utility
Rule: prefill 1200, decode 726	750.0	144.5	-0.2845
Bandit: flat 902	589.5	181.4	-0.2440
Difference	**-160.5 (-21%)**	**+36.9 (+26%)**	**+0.0405 (+14%)**

The rule refuses that plan because 181.4 s misses the mid tier's 173.1 s time budget, not because it ranks it lower.

TABLE 9 · WHAT LEARNING COST

Regret against an oracle playing the bandit's own best arm

	Pulls	Energy (kJ)	Regret
Warm start, one pull per context and arm	9	5.9	0.542
Free choices	15	9.8	0.043
Total	**24**	**15.7**	**0.585**

15.7 kJ
energy to learn

149 min
phone time

- 93% of the regret falls in the mandatory warm start, which must play arms it already has reason to avoid.
- The rule pays none of this. Its table is measured once, offline, and then corrected by two loops at 0.3 s per request.
- So learning is worth it only where the rule's time guard binds, and only if you are willing to trade 26% latency for 21% energy.

RELATED WORK

Nobody has measured KV eviction on a phone GPU

System	Venue	Hardware evaluated on	Phone	Compute	KV mechanism
KV EVICTION, BUT NEVER ON A PHONE					
SnapKV, Ada-KV, H2O, TOVA, PyramidKV	2023-24	A100-class servers	no	server GPU	eviction
KVSwap	MobiSys 2025	Jetson Orin AGX 64 GB + NVMe	no	board GPU	offload to disk
MobiLoRA	ACL 2025	NVIDIA AGX Orin	no	board GPU	cache reuse
KeyDiff	NeurIPS 2025	A100; unnamed Samsung phone (kernel only)	yes	not stated	eviction
ON A REAL PHONE, BUT NEVER EVICTION, NEVER THE GPU					
DynaKV	2025	OnePlus Ace5 Pro, 12, Ace3, Ace2	yes	CPU only	offload to UFS
ActiveFlow	2025	OnePlus 12, Pixel 6, Infinix ZERO 30	yes	CPU only	weights, not KV
PowerInfer-2	MobiSys 2025	OnePlus 12, OnePlus Ace 2	yes	CPU and NPU	neurons, not KV
PerCache	2026	Pixel 7, Redmi K60 Pro, S22 Ultra	yes	not stated	QKV reuse (RAG)
ON A PHONE GPU, BUT NO CACHE MANAGEMENT					
LLMs in Your Pockets	2026	7 devices: Adreno 750/730, Mali G720/G78	yes	CPU, GPU, NPU	none
LLM Inference at the Edge	2026	S24 Ultra (Adreno 750), iPhone 16 Pro	yes	GPU	none
µKV (this work)		**OnePlus 15, Adreno 840**	**yes**	**CPU and GPU**	**eviction + compaction**

Stated plainly, because a workshop paper should

- Eviction is destructive. μKV cannot recall a span it dropped.
- The natural next step is a third tier: park mid-scored cells in NAND using the scores μKV already computes. Reload is 23 MB for 721 cells, single-digit ms at UFS 4.0.
- The blocker is not reload cost. No mobile engine can shrink and regrow its KV pool, so parking frees nothing today.
- Nobody has measured what parking costs in flash write energy and wear. That is the number that decides whether the tier is worth having.
- The ladders are calibrated to one SoC. What generalizes is the protocol of anchoring a ladder above a measured vendor trigger, not the thresholds.
- Four of the per-head GPU cells are n = 1. The CPU StreamingLLM row still carries our fused-kernel omission.

TAKEAWAYS

Budgets are not compression
A per-head budget that looks good on a server is a full cache in disguise on the device. 36 to 68% stays live.

The kernel path sets the speed
Reading attention scores costs FlashAttention. Four of five published policies decode slower than not evicting.

The axis of selection is the thing
Sequence-level compacts and keeps the kernel. A baseline we did not author confirms it.

µKV is the existence proof that one policy can hold all three properties at once: in-graph scoring, one keep set, in-place compaction.
On the phone it decodes 4.8x the full cache on the CPU at 7% of the cells and 62% less energy, and 1.19x on the Adreno at the same cell count.

Report realized cells, not selection ratios. Say which kernel your policy leaves behind.